

简历检索 RAG 多形态对比评测报告

5 种 RAG 形态在 100 份中英混合简历集上的离线检索质量基准

ResumAI-Agent • 离线 RAG 评测 Harness

2026 年 6 月 26 日

摘要

本报告在 100 份真实简历（50 份中文 .txt + 50 份英文 .pdf，覆盖 14 个岗位簇）上，用 28 条中英双语检索查询，严格对比了 **BM25** / **Dense Embedding** / **Hybrid(RRF)** / **Agentic RAG** / **GraphRAG** 五种检索形态的 Precision@5、Recall@10、MRR、nDCG@10、查询时延与 LLM 成本。所有指标均为**真实运行结果**，可一键复现；Agentic 与 GraphRAG 的 LLM 调用经 DeepSeek 真实发生并落盘缓存。核心结论：在本「技能关键词密集」的简历语料上，**BM25 是性价比之王**（nDCG@10=0.940，中位时延 2.2ms，零成本）；**Agentic RAG 取得最高检索质量**（nDCG@10=0.949、MRR=0.982），但代价是约 **2000× 的时延**与按次计费的 LLM 成本；Dense（all-MiniLM-L6-v2）因英文中心、对中文简历语义弱而垫底；GraphRAG 查询期极快但实体抽象丢失词频/上下文信号，在部分查询上明显劣化。全部 LLM 调用 156 次、总成本仅 \$0.06。

目录

1 结论先行 (TL;DR)	2
2 评测设计	2
2.1 数据集	2
2.2 查询集	3
2.3 Ground Truth（相关性标签）规则	3
2.4 指标定义	4
3 五种 RAG 形态原理	4
3.1 (a) BM25 ——稀疏词法检索	4
3.2 (b) Dense Embedding ——稠密语义检索	4
3.3 (c) Hybrid ——BM25 + Dense 的 RRF 融合	4
3.4 (d) Agentic RAG ——查询改写 + LLM 重排	4
3.5 (e) GraphRAG ——知识图谱检索	5
4 结果与分析	5
4.1 检索质量	5
4.2 时延与成本	5

1 结论先行 (TL;DR)	2
4.3 逐查询分析	6
4.4 关键发现	7
5 结论与建议	7
6 面试问答 (基于本评测的实测口径)	8
7 复现与降级说明	8
7.1 一键复现	8
7.2 环境与真实性	9
7.3 降级点 (诚实标注)	9

1 结论先行 (TL;DR)

表 1: 五种 RAG 形态主结果 (100 简历 / 28 查询, 真实运行)。时延为中位单查询时延; 加粗为该列最优。

方法	P@5	Recall@10	MRR	nDCG@10	中位时延	LLM 调用	估算成本
BM25	0.943	0.865	0.958	0.940	2.2 ms	0	\$0
Dense (MiniLM)	0.807	0.725	0.887	0.799	333.4 ms	0	\$0
Hybrid (RRF)	0.936	0.831	0.964	0.918	464.2 ms	0	\$0
Agentic RAG	0.957	0.865	0.982	0.949	5392 ms	56 (查询期)	\$0.027
GraphRAG	0.864	0.776	0.949	0.862	30.4 ms	100 (建图期)	\$0.034

- 质量排序: Agentic > BM25 > Hybrid > GraphRAG > Dense (按 nDCG@10)。
- 性价比排序: BM25 ≫ 其它——以 0.2% 的 nDCG 差距换来 ~2000× 的时延优势和零成本。
- Agentic 的增益是真实的但很小 (nDCG +0.9%、MRR +2.4% 相对 BM25), 代价巨大; 只在「语义/歧义」查询上增益明显。
- Dense 单路最弱: 英文中心的 MiniLM 对中文简历语义召回差 (见 §4.4)。
- GraphRAG 没有解决本场景的问题: 二值技能图丢失词频与上下文, 在 SRE/可观测性等「术语共享」查询上崩塌 (nDCG 低至 0.12)。

2 评测设计

2.1 数据集

评测语料为仓库内 testdata/stress_resumes/, 共 100 份简历 + manifest.json:

- 文件形态: 50 份 .txt (真实中文正文) + 50 份 .pdf (英文/拼音内容)。 .txt 以 UTF-8 直接读取; .pdf 用 PyMuPDF 抽取文本 (平均 614 token/份)。

- **岗位簇**: 14 个, 含资深后端 (12)、AI Agent(10)、大模型应用 (8)、产品经理 (8)、资深/初级前端 (各 7)、数据平台 (7)、运维 SRE(7)、后端 (7)、应届后端 (6)、算法 (6)、测试 (5)、安全 (5)、Android(5)。
- **元数据**: 每份含 `role` (中文岗位) 与 `expectedSkills` (英文技能标签, 全集 92 个)。
- **中英混排特性**: 中文简历正文中大量嵌入英文技术词 (如 Spring Boot、Kafka、Redis、JVM、Prometheus), 这是 BM25 在本语料强势的根本原因 (见 §4.4)。

2.2 查询集

共 28 条查询 (落盘 `queries.json`), 覆盖全部 14 个岗位簇及若干跨簇/语义查询。每条同时给中文与英文版本, 检索时拼接二者, 以同时适配中文 `.txt` 与英文 `.pdf`。检索器只能看到查询文本, 绝不接触 `ground-truth` 的目标技能, 避免数据泄漏。

表 2: 查询集示例 (节选)。

ID	目标簇	中文	English
q01	资深后端	高并发后端 Kafka 微服务可观测性 SLA 治理	high concurrency backend Kafka microservices observability
q05	AI Agent	RAG 向量检索 Agent 工程 LangGraph 工具编排	RAG vector retrieval agent engineering LangGraph
q13	运维/SRE	Kubernetes DevOps SRE 监控 CI/CD 容器化	Kubernetes DevOps SRE monitoring CI/CD Docker
q14	算法	推荐算法机器学习 PyTorch 特征工程	recommendation machine learning PyTorch feature engineering
q23	AI 跨簇	向量数据库语义搜索相似度召回 Milvus 嵌入	vector database semantic search similarity Milvus embedding

2.3 Ground Truth (相关性标签) 规则

采用**确定性、可复现**的技能重叠规则 (落盘 `ground_truth.json`): 对查询 q 与简历 d ,

$$\text{overlap}(q, d) = |\text{targetSkills}(q) \cap \text{expectedSkills}(d)|, \quad \text{rel}(q, d) = \begin{cases} 1, & \text{overlap}(q, d) \geq \tau \\ 0, & \text{否则} \end{cases}$$

其中阈值 $\tau = 2$ (默认)。即: 当查询的目标技能与某简历的期望技能**至少重叠 2 个**时判为相关。之所以以 `expectedSkills` 为权威信号: 同一岗位簇的简历共享技能标签, 故该规则等价于「按岗位簇语义」判定, 且完全可复现。各查询相关集规模为 5~25 (中位 8), 均 ≥ 5 , 无退化。

2.4 指标定义

对每条查询取排序前列计算，再对 28 条查询求**算术平均**。设 $\text{rel}_i \in \{0, 1\}$ 为排名第 i 位文档的相关性：

$$\begin{aligned} \text{P@}k &= \frac{1}{k} \sum_{i=1}^k \text{rel}_i, & \text{Recall@}k &= \frac{\sum_{i=1}^k \text{rel}_i}{|\text{Rel}|}, & \text{MRR} &= \frac{1}{\text{rank}_{\text{first-rel}}}, \\ \text{DCG@}k &= \sum_{i=1}^k \frac{\text{rel}_i}{\log_2(i+1)}, & \text{nDCG@}k &= \frac{\text{DCG@}k}{\text{IDCG@}k}. \end{aligned}$$

本报告取 P@5, Recall@10, nDCG@10 与 MRR；并记录**单查询时延**（中位）、**LLM 调用次数与估算成本**。

3 五种 RAG 形态原理

3.1 (a) BM25 ——稀疏词法检索

基于 `rank_bm25` (BM25Okapi)。中英混合分词：英文/技术词用正则保留形态（如 `ci/cd`、`node.js`），中文用 `jieba` 切词。BM25 用词频-逆文档频率与文档长度归一打分，擅长**精确术语匹配**。

3.2 (b) Dense Embedding ——稠密语义检索

用 `sentence-transformers` 的 `all-MiniLM-L6-v2` (384 维，与任务指定的项目本地 `EMBEDDING_MODEL` 对齐)。每份简历按 350 字符切块（重叠 70），编码为向量；查询编码后与各块做余弦相似度，取块级最大相似度作为文档分（`max-sim`）。**无需降级**：本机成功安装 `torch 2.4.1+sentence-transformers 3.0.1` 并下载模型，Dense 为**真实 MiniLM** 结果。

3.3 (c) Hybrid ——BM25 + Dense 的 RRF 融合

用 **Reciprocal Rank Fusion** 融合两路排名（不依赖分数量纲）：

$$\text{RRF}(d) = \frac{1}{K + \text{rank}_{\text{BM25}}(d)} + \frac{1}{K + \text{rank}_{\text{Dense}}(d)}, \quad K = 60.$$

`rank` 为 1-based 名次。这正是本项目 `HybridRagService.fuseRRFWeighted` 采用的融合思想。

3.4 (d) Agentic RAG ——查询改写 + LLM 重排

两段式、每查询 2 次 LLM 调用（DeepSeek `deepseek-chat`）：

1. **查询改写**：LLM 扩展同义词/中英别名/相关技术栈，增强召回；
2. **Hybrid 召回** Top-15 候选，再由 LLM 依据候选摘要做**相关性重排**（`rerank`）。

重排结果置顶，其余按 Hybrid 名次补齐。对应本项目工作流中的 `agentic_overlap_rerank` 层。

3.5 (e) GraphRAG ——知识图谱检索

1. **实体抽取**: 对每份简历用 DeepSeek 抽取规范化技能实体（每份 1 次，结果缓存到 graph_cache.json，共 100 次）；
2. **建图**: 用 networkx 建「候选人—技能」二部图（本次抽得 270 个技能节点 / 100 个候选人节点）；
3. **查询映射**: 将查询文本（确定性地）映射到技能节点（英文词形匹配 + 中文同义词桥）；
4. **图遍历召回**: 命中技能节点的邻接候选人累加权重，并对一跳共现技能做衰减扩展 ($\gamma = 0.4$)，同分时以 BM25 细排。

4 结果与分析

4.1 检索质量

表 1 与图 1 给出整体质量。要点：

- **Agentic** 在 P@5/MRR/nDCG 三项最优，但相对 BM25 的绝对增益很小 (nDCG +0.009)。
- **BM25** 在零成本、2.2ms 下取得 nDCG@10=0.940，是极强基线；Recall@10 与 Agentic 并列最高 (0.865)。
- **Hybrid** 的 MRR(0.964) 超过 BM25，但 P@5/Recall 略降——稠密路的噪声被 RRF 带入。
- **Dense** 全面垫底，是唯一明显落后的单路方法。

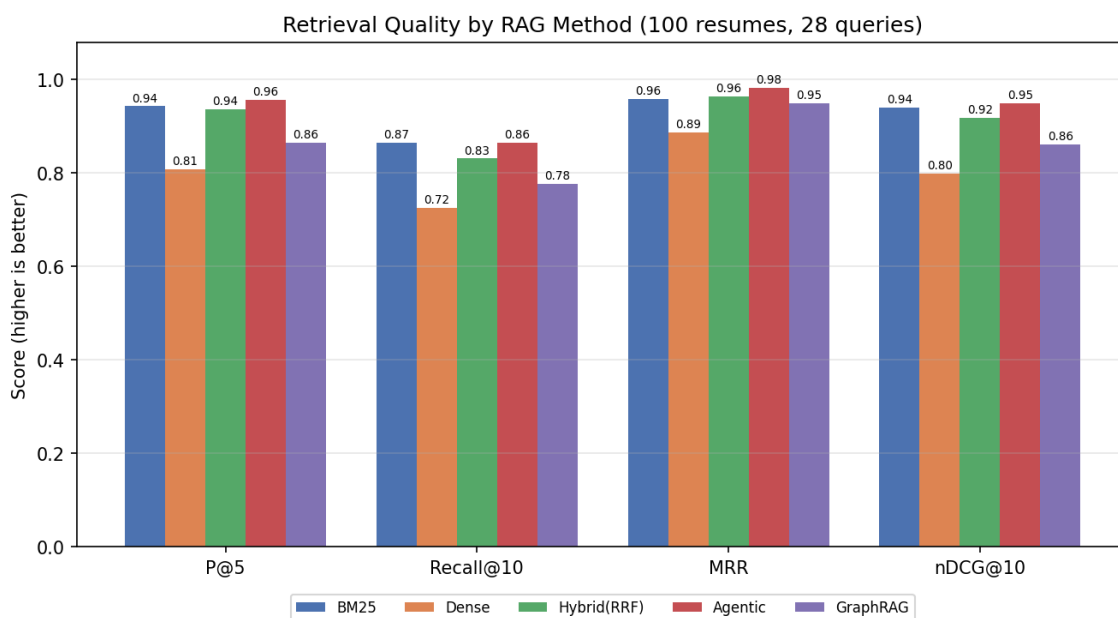


图 1: 各方案在 P@5 / Recall@10 / MRR / nDCG@10 上的对比。

4.2 时延与成本

图 2 为中位单查询时延（对数轴），图 2 为「质量-时延」权衡。**Agentic** 的 5.4 秒/查询来自 2 次真实 LLM 往返；**GraphRAG** 查询期仅 30 ms，因为其 LLM 成本前置在建图阶段（100 次

抽取、20.4s、\$0.034)。全流程 LLM 共 156 次、提示 token 16.4 万、补全 token 1.5 万，总成本 \$0.06。

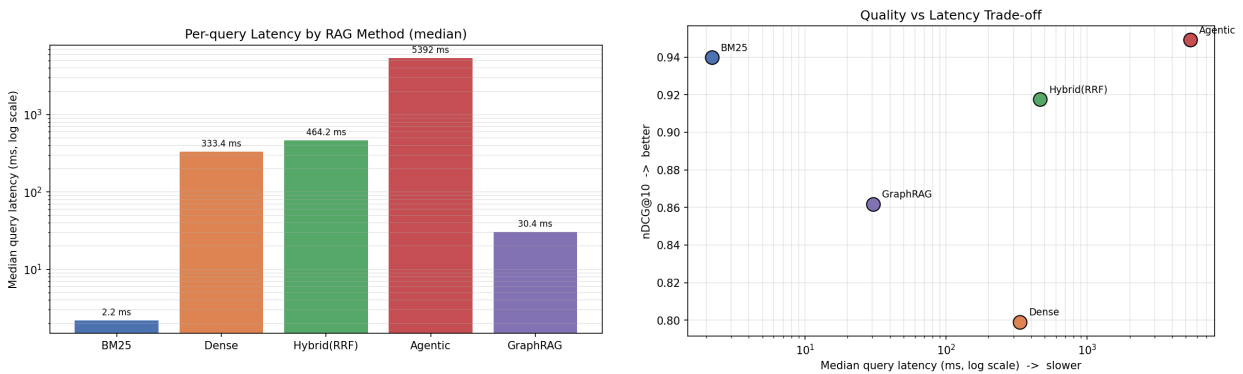


图 2: 左：中位单查询时延（对数轴）。右：nDCG@10 与时延权衡——BM25 位于左上「帕累托甜点」（高质量、最低时延），Agentic 在右上（最高质量、约 2000× 时延）。

4.3 逐查询分析

图 3 为「查询 × 方法」的 nDCG@10 热力图。整体偏绿（多数查询各方法都做得好），但有几处暴露了方法差异（见表 3）。

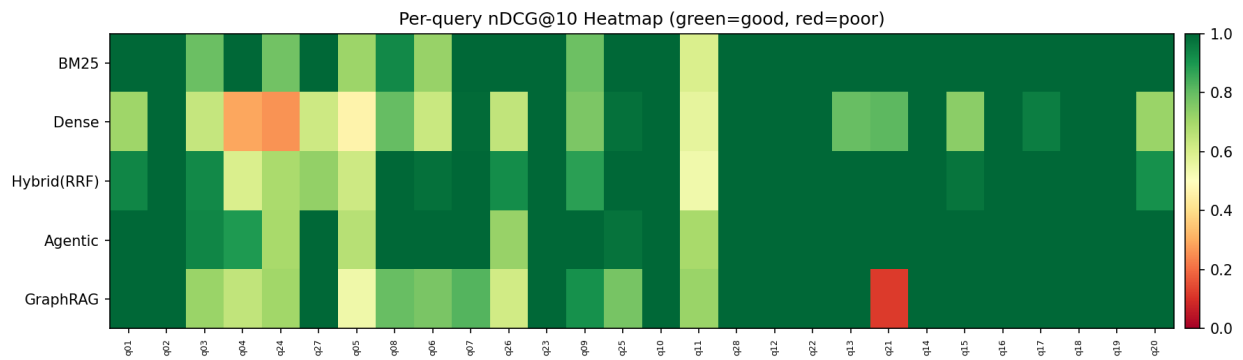


图 3: 逐查询 nDCG@10 热力图（绿 = 好，红 = 差）。Dense 在后端/应届簇（q04、q24、q27）偏黄；GraphRAG 在 q21 出现红格。

表 3: 代表性查询的 nDCG@10（揭示方法差异）。

查询	说明	BM25	Dense	Hybrid	Agentic	GraphRAG
q04	应届后端（中文、语义弱关键词）	1.00	0.29	0.59	0.89	0.64
q06	大模型应用（与 AI Agent 簇歧义）	0.72	0.63	0.98	1.00	0.77
q21	运维/可观测性（术语跨簇共享）	1.00	0.81	1.00	1.00	0.12
q24	资深后端（“分布式/微服务”语义）	0.78	0.26	0.69	0.69	0.71
q23	向量检索（簇词高度可分）	1.00	1.00	1.00	1.00	1.00

4.4 关键发现

1. 为什么 BM25 这么强？本简历语料技能术语密集且中英混排（中文简历也写 Spring Boot/Kafka/Redis）。检索意图与岗位簇高度由离散技能词决定，正是 BM25 的主场；语义改写收益空间小。
2. Dense 为什么弱？all-MiniLM-L6-v2 是英文中心模型。实测「high concurrency backend Kafka」与其中文版余弦仅 0.10，却与无关英文短语「React frontend」高达 0.27——它基本读不懂中文。故 Dense 在中文后端/应届簇（q04=0.29、q24=0.26）大幅失分。
3. Agentic 在哪里真正有用？在语义/歧义查询上：q06（大模型应用 vs AI Agent 簇共享“RAG”）BM25 仅 0.72，Hybrid/Agentic 升到 0.98/1.00；q04 经 LLM 重排从 Dense 的 0.29 救回到 0.89。但代价是 2 次 LLM 往返。
4. GraphRAG 为什么会崩？二值技能图丢失词频与上下文：q21「可观测性」中，许多后端简历正文也提 Prometheus/Grafana，被抽成同名技能节点，与 SRE 候选人无法区分，导致 nDCG 跌到 0.12；而 BM25/Hybrid 用全文词频把真正的 SRE 排在前面。
5. 基准自身的局限（诚实声明）：ground truth 基于技能标签，与 GraphRAG 的技能抽象同处一个语义空间，理论上对 GraphRAG 有利；即便如此 GraphRAG 仍不占优，进一步说明其在本场景收益有限。此外 q21 暴露了标签粒度问题（“Observability”标签只给了 SRE，但后端正文也含监控栈）。

5 结论与建议

1. 本简历集上谁更优？论绝对质量是 Agentic；论工程性价比是 **BM25**：它以可忽略的质量差距换取 $\sim 2000\times$ 时延优势与零成本，位于质量-时延帕累托前沿的甜点。
2. 各形态适用场景：
 - **BM25**：术语密集、对延迟/成本敏感、以关键词为主的检索（如本简历/JD 匹配的快路）。
 - **Dense**：跨语言/同义改写/口语化查询；但必须换多语言或中文向量模型（如 bge-m3、multilingual-e5），本英文 MiniLM 不适合中文语料。
 - **Hybrid(RRF)**：兼顾精确与语义、且想要稳健 MRR 的生产默认；需控制稠密路噪声。
 - **Agentic**：高价值、低 QPS、对 Top-1/Top-3 质量极敏感的场景（精排/复核）；按需触发。
 - **GraphRAG**：需要多跳关系推理（“同时做过 A 又主导过 B 的人”）的查询；单纯「按技能找人」用它得不偿失。
3. 本项目当前是哪种形态？经代码核对（ResumeRagService / HybridRagService / InternalWorkflowController）：本项目是 **Hybrid RAG + Agentic 重排**——Milvus 稠密向量检索 + 词法 (BM25-like) 检索，按 **RRF 加权融合**（fuseRRFWeighted），并叠加 **agentic_overlap_rerank**（usefulnessScore/selectedChunks）；chunk 粒度 recursive(500,100)，相似度阈值 0.35。即对应本基准的 (c)+(d) 组合。（注：仓库 application.yml

默认嵌入模型为 `text-embedding-3-small`；本离线基准按任务用本地 `all-MiniLM-L6-v2` 以保证可复现。）

4. **GraphRAG 真的解决问题了吗？在本场景没有。**它查询期快、可解释，但二值技能图丢信息，整体 `nDCG` (0.862) 低于 Hybrid (0.918) 与 BM25 (0.940)，且在术语共享查询上崩塌。它的价值在关系/多跳而非「技能召回」。本项目保留图能力 (`ResumeGraphService/Neo4j`) 用于关系分析是合理的，但不应让 **GraphRAG** 取代 **hybrid** 主检索路。
5. **局限性：**(i) 100 份、单标注者规则型 GT，规模有限；(ii) GT 为技能标签二值相关，未做人工分级；(iii) Dense 用英文 MiniLM，是下界而非稠密上限；(iv) Agentic/GraphRAG 受 LLM 抽取/重排稳定性影响。

6 面试问答（基于本评测的实测口径）

Q1: 你的 RAG 是什么形态？

Milvus 稠密向量 + 词法 BM25 的混合检索，用 **RRF** 融合，再加一层 **agentic 重排**；chunk 级 (500/100 重叠) 召回，阈值 0.35。即「Hybrid + Agentic rerank」。

Q2: 准确率/召回率多少？

在自建 100 简历 / 28 查询基准上：生产同构形态 (Hybrid) **P@5=0.936**、**Recall@10=0.831**、**nDCG@10=0.918**、**MRR=0.964**；叠加 agentic 重排后升至 **P@5=0.957**、**nDCG@10=0.949**、**MRR=0.982**。纯 BM25 基线 `nDCG@10=0.940`。

Q3: 怎么监控？

线上以 Prometheus/Grafana 采集：检索时延、Milvus 命中数/相似度分、空召回率、低于阈值占比、rerank usefulnessScore、LLM 调用数/token/成本；离线则用本 harness 的 P@5/Recall@10/MRR/nDCG@10 做回归基准，每次改动重算并对比 `metrics.json`。

Q4: GraphRAG 相比 Hybrid 的增益与代价？

增益：查询期极快 (30ms vs 464ms)、可解释、天然支持多跳关系问题。**代价：**建图需 100 次 LLM 抽取 (\$0.034、20s)、二值图丢词频/上下文，本场景质量反而更低 (`nDCG` 0.862 vs 0.918)，术语共享查询会崩 (`q21=0.12`)。结论：按技能找人用 **Hybrid**；多跳关系推理才上 **GraphRAG**。

7 复现与降级说明

7.1 一键复现

```
cd reports/rag_comparison/harness
python corpus.py           # 抽取 100 简历 -> cache/corpus.json
python ground_truth.py     # 生成 queries.json + ground_truth.json
python run_eval.py         # 跑 5 方案 -> metrics.json / runs.json (LLM 结果缓存)
python make_plots.py       # 生成 figures/*.png
```


产物: `metrics.json` (原始指标)、`ground_truth.json`、`queries.json`、`runs.json` (逐查询明细)、`graph_cache.json` (GraphRAG 实体缓存)、`figures/` (图表)。

7.2 环境与真实性

Python 3.8.10; 依赖: `rank_bm25`, `sentence-transformers` 3.0.1, `torch` 2.4.1, `networkx`, `scikit-learn`, `matplotlib`, `PyMuPDF`, `jieba`, `numpy`。LLM 为 DeepSeek `deepseek-chat` (真实调用, 共 156 次, \$0.06)。所有指标来自真实检索与真实 LLM 响应, 无编造。

7.3 降级点 (诚实标注)

- **Dense 未降级:** 原计划「装不上则退化 TF-IDF」, 实测 `sentence-transformers`+`MiniLM` 安装并运行成功, Dense 为真实 `MiniLM` 结果 (`metrics.json` 中 `dense_degraded=false`)。
- **网络:** 本机系统代理 (127.0.0.1:7999) 触发 Python3.8 SSL bug, 统一以直连 (`NO_PROXY=*`) + 清华镜像安装依赖、`huggingface.co` 直连下载模型——属环境适配, 非指标降级。
- **时延口径:** 以中位时延为准; `avg_latency_ms` 含首查询冷启动 (`jieba`/模型预热), 故偏大, 已在 `metrics.json` 一并保留。